



POSETTE:
An Event for Postgres

2026

pg_duckdb in Action: Accelerating Analytics on Azure Database for PostgreSQL



Nitin Jadhav

Hosted by



Microsoft

| June 16-18, 2026 • posetteconf.com

Agenda

1



What is pg_duckdb?

2



PostgreSQL's Strengths

3



Limitations for Analytics

4



Introducing DuckDB

5



PostgreSQL vs DuckDB

6



How PostgreSQL Processes Queries

7



How pg_duckdb Works

8



What pg_duckdb Can Do

9



Demo · pg_duckdb on Azure Database for PostgreSQL

10



Key Takeaways

What is pg_duckdb?



Bringing DuckDB's Analytical Power inside PostgreSQL.

*The Best of Both Worlds:
OLTP + OLAP.*

PostgreSQL's Strengths (1 of 2)

The world's most trusted open-source relational database.

ACID Compliance



Robust ACID compliance and data integrity for mission-critical applications.

Rich Extensibility



Custom types, operators, and extensions through a powerful extensibility model.

Open-Source Ecosystem



Thriving community support and a broad open-source ecosystem.

PostgreSQL's Strengths (2 of 2)

Battle-Tested OLTP



Proven at scale for transactional workloads with high concurrency.

Extensive Indexing



B-tree, GIN, GiST, and BRIN indexes for diverse query patterns.

Mission-Critical Ready



Powering enterprise and mission-critical applications worldwide.

OLTP vs OLAP

OLTP

Online Transaction Processing

- INSERT, UPDATE, DELETE focused
- Optimized for low latency per query
- Row-oriented storage
- High concurrency (many simultaneous users)

OLAP

Online Analytical Processing

- SELECT with aggregations (SUM, AVG, COUNT)
- Optimized for throughput over latency
- Columnar storage
- Large scans over millions or billions of rows

A million single-row queries vs. a single query over a million rows

PostgreSQL's Limitations for Analytics

No Columnar Storage

Reads entire rows even when only a few columns are needed — wastes I/O on analytical scans.

No Vectorized Execution

Processes data one row at a time, missing CPU cache and SIMD efficiency.

Limited Parallelism

Parallel query exists but is constrained vs. purpose-built OLAP engines.

Why PostgreSQL Struggles with OLAP

Query: `SELECT SUM(revenue) FROM orders;`

Row-based (PostgreSQL)

ID	Name	City	Revenue

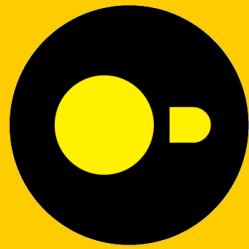
Reads ALL columns for every row

Columnar (DuckDB)

ID	Name	City	Revenue

Reads ONLY the column needed

INTRODUCING



DuckDB

The in-process analytical SQL database.

What Makes DuckDB Special



In-Process Analytics Engine

Runs embedded inside your application — no separate server needed.



Columnar + Vectorized Execution

Processes data in column batches for maximum CPU cache utilization.



Parallel Execution

Automatically parallelizes queries across all available CPU cores.



Native Data Lake Support

Access Parquet, CSV, JSON, and Iceberg directly — local or in object storage.

PostgreSQL vs DuckDB (1 of 2)

Two databases — built for very different jobs.

P PostgreSQL	VS	DuckDB D
OLTP — transactions	Workload	OLAP — analytics
Row-oriented	Storage Model	Columnar
Row-at-a-time (Volcano)	Execution	Vectorized — batched
High — MVCC, many users	Concurrency	Single-query optimized

PostgreSQL vs DuckDB (2 of 2)

Different parallelism, data-source, and architecture trade-offs.

P

PostgreSQL



DuckDB

D

Limited parallel query

Parallelism

Fully parallel across cores

Local tables

Data Sources

Parquet, CSV, JSON, Iceberg

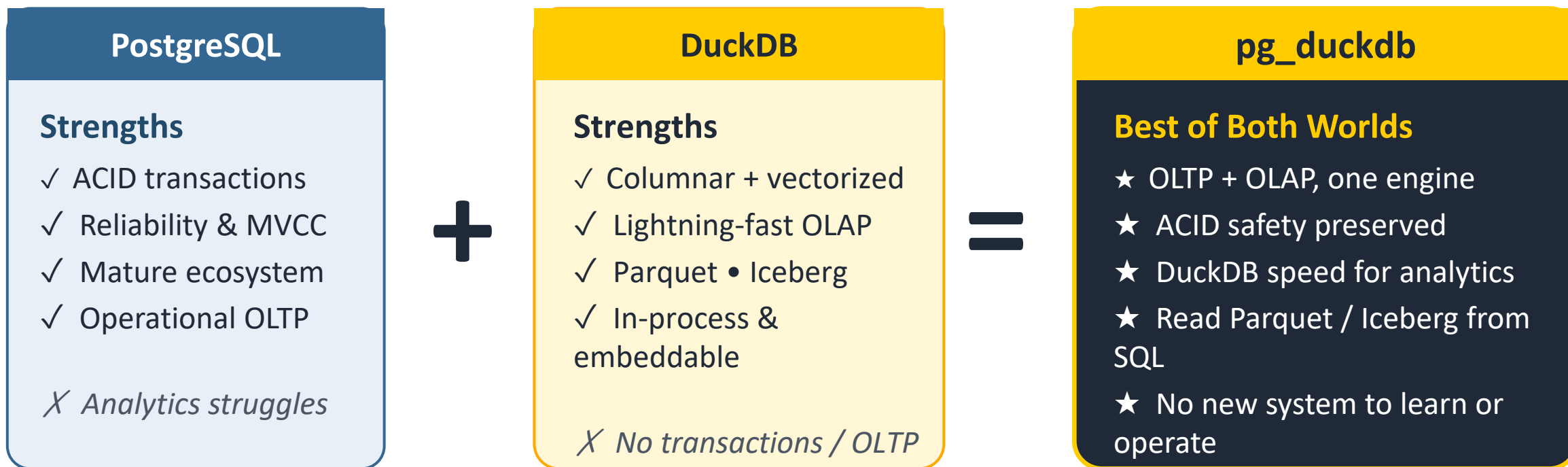
Client-server

Architecture

In-process (embedded)

pg_duckdb: The Best of Both Worlds

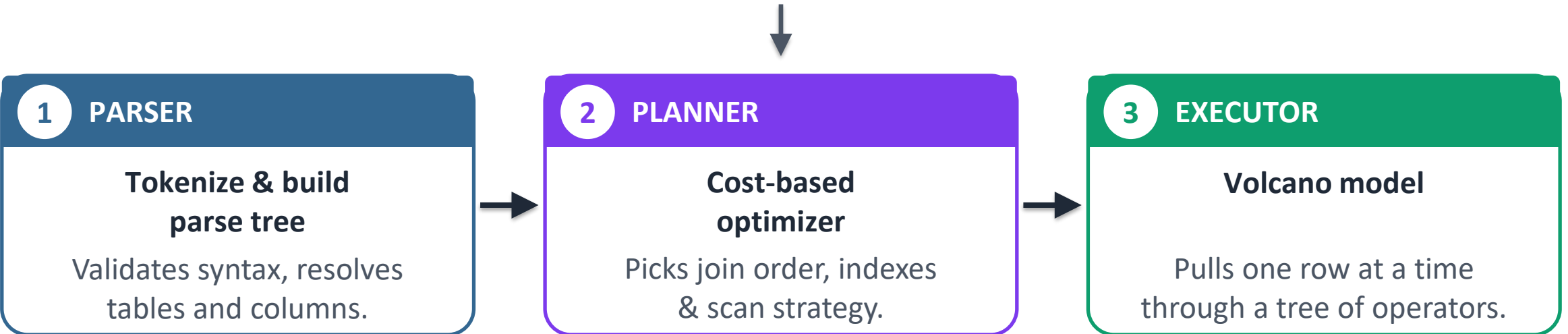
PostgreSQL's reliability + DuckDB's analytical power — in one engine.



How PostgreSQL Processes a Query

From SQL text to result rows — the classic PostgreSQL pipeline.

```
> SELECT customer_id, SUM(amount) FROM orders WHERE region = 'EU' GROUP BY  
customer_id;
```

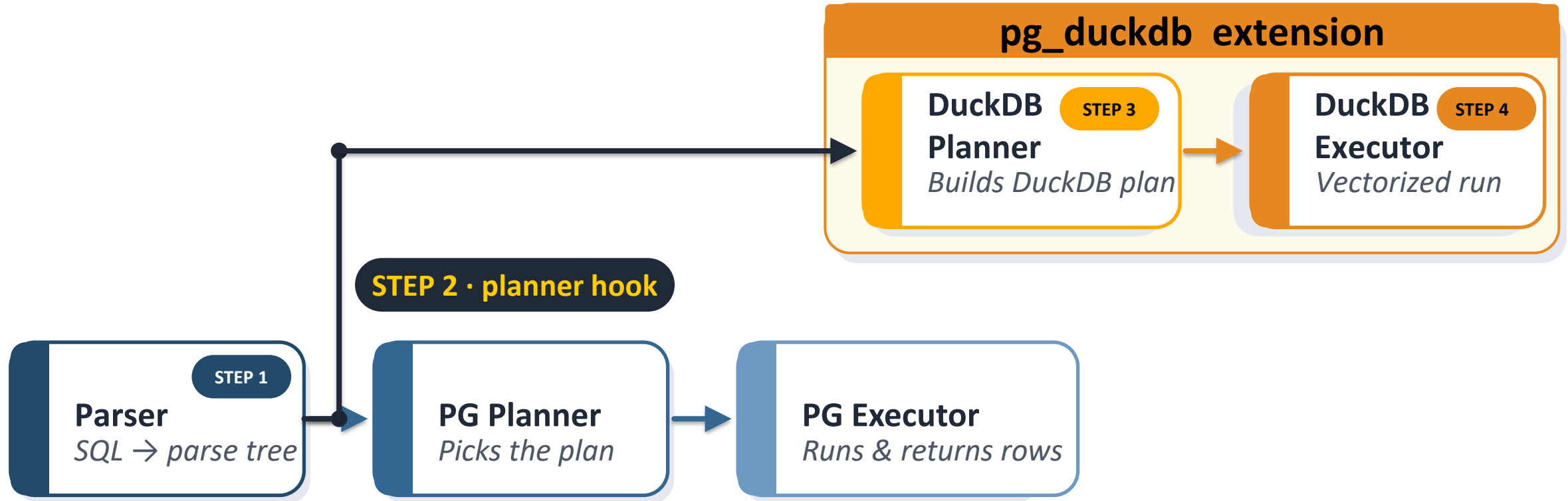


Row-by-row execution.

Great for OLTP point queries — the bottleneck for analytical scans.

How pg_duckdb Works

A planner hook diverts the parse tree to DuckDB.



Configuration decides the route — see the next slide for what triggers DuckDB.

When Does DuckDB Take Over?



duckdb.force_execution = on

All eligible SELECT queries are routed to DuckDB automatically.



Query reads data-lake files

Using `read_parquet()`, `read_csv()`, or `read_json()` functions.



Automatic fallback

If DuckDB cannot handle the query, execution falls back to PostgreSQL transparently.

What pg_duckdb Can Do

Three superpowers for analytics on Azure Database for PostgreSQL.

1 Accelerate PG Tables

Run DuckDB's vectorized engine on tables in Azure Database for PostgreSQL.

```
SET duckdb.force_execution = on;  
SELECT region, SUM(amount)  
FROM orders GROUP BY region;
```

2 Read & Write Azure Blob

Query Parquet, CSV, JSON directly from Azure Blob Storage.

```
SELECT customer_id, SUM(amount)  
FROM  
read_parquet('az://lake/orders/*');
```

3 Hybrid Analytics

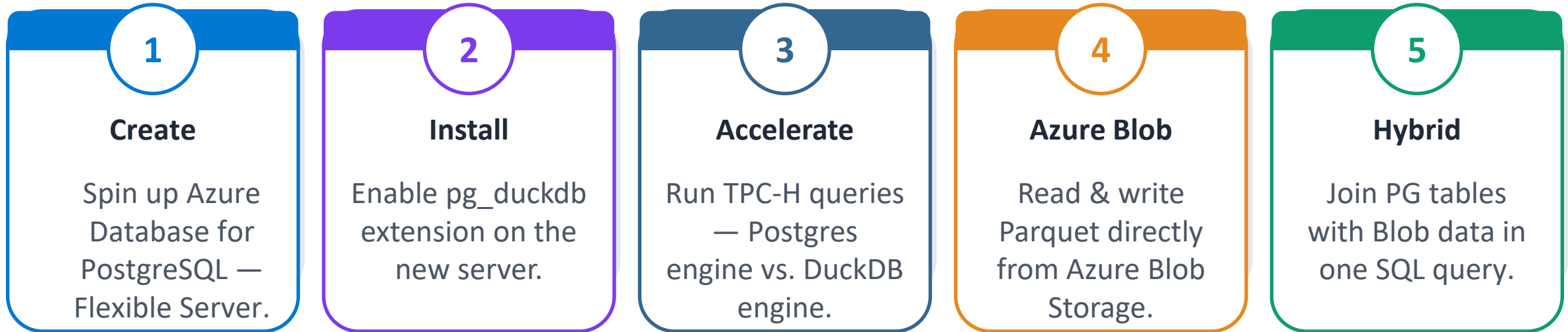
Join live PG tables with Azure Blob data in one SQL query.

```
SELECT c.name, SUM(o.amount)  
FROM customers c JOIN  
read_parquet('az://history/*') o  
ON c.id = o.customer_id;
```

Accelerate your tables. Reach into Azure Blob. Join them — all from plain SQL.

Demo - pg_duckdb on Azure Database for PostgreSQL

End-to-end: provision → install → accelerate → blob storage → hybrid.



WHAT TO WATCH FOR

Query time

Postgres vs DuckDB on the same TPC-H query.

Plan switch

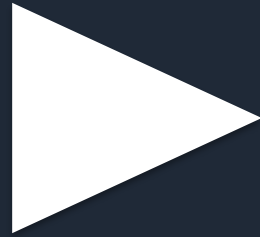
When duckdb.force_execution kicks in.

Same SQL

Identical query — only the engine changes.

Same managed Postgres. Same SQL. Faster analytics.

Demo - pg_duckdb in action



DEMO VIDEO

pg_duckdb on Azure Database for PostgreSQL

Key Takeaways

1

One database. Two engines.

Postgres for OLTP. DuckDB for OLAP.

3

Hybrid analytics.

Join PG tables and Blob files in one SQL.

2

No data movement.

Query Parquet on Azure Blob directly.

4

Same SQL. Faster answers.

Opt-in via `duckdb.force_execution`.

Thank You



Questions & feedback welcome.

Nitin Jadhav · Microsoft · POSETTE 2026